

Empirical Study on Engineering Decisions in Multi-Agent Reinforcement Learning Systems

Deepansha Chowdhary¹(✉)[0009–0008–9786–6164] and Uwe
Zdun²[0000–0002–6233–2591]

¹ UniVie Doctoral School Computer Science (DoCS), University of Vienna, Vienna,
Austria

`deepansha.chowdhary@univie.ac.at`

² Faculty of Computer Science, Research Group Software Architecture, University of
Vienna, Vienna, Austria
`uwe.zdun@univie.ac.at`

Abstract. Multi-agent reinforcement learning (MARL) is increasingly used in real-world coordination tasks, yet little is known about how MARL systems are engineered in practice. We present an empirical repository-mining study of 187 open-source GitHub repositories comprising 12,205 files (9,957 Python, 2,090 YAML, and 158 JSON). Using automated static analysis and manual repository screening, we examine engineering decisions in algorithm adoption and hyperparameter configuration, environment integration, and multi-agent implementation patterns. Our results show that open-source MARL development is concentrated around a small set of algorithm families, especially PPO, DQN and QMIX, and that algorithm use co-occurs with recurring library and environment combinations. Hyperparameter practice is largely limited to a small set of recurring defaults, while specialized variants tend to involve larger hyperparameter sets than their base algorithms. Repositories most often realize multi-agent behavior through simple structural patterns, such as instantiating multiple agents or defining multiple agent classes, whereas specialized multi-agent abstractions occur less frequently. These findings provide an empirical baseline for current MARL engineering decisions and highlight opportunities to improve standardization in hyperparameter specification, environment integration, and multi-agent abstractions.

Keywords: Reinforcement Learning · Multi-Agent Systems · Repository Mining · Empirical Software Engineering · RLOps

1 Introduction

Multi-agent reinforcement learning (MARL) [4] is increasingly used to address coordination problems in domains such as production automation, autonomous driving [24], smart grids [21], and robotics [17]. Alongside this growth, an ecosystem of frameworks, libraries, and tools has emerged to support MARL development, including RLLib [14], Stable-Baselines3 [19], and BenchMARL [3]. These

frameworks provide templates, defaults, and environment integrations, but they do not by themselves show how practitioners combine algorithms, libraries, environments, and multi-agent patterns across projects.

This creates an important empirical gap. Although prior work has studied MARL algorithms, benchmarks, and theoretical properties [25,20,7,10], much less is known about engineering decisions in open-source MARL systems. In particular, we lack cross-project evidence on which algorithms are adopted in practice, how they are configured, which environments and libraries they are combined with, and how multi-agent functionality is realized in code. These questions matter for reproducibility because many MARL engineering decisions, including hyperparameters, environment wrappers, and implementation structure, are only partially documented in papers and tutorials [8,15,16].

To address this gap, we conduct an empirical repository-mining study of open-source MARL projects on GitHub. We examine how existing methods are engineered in practice, focusing on algorithm adoption, hyperparameter configuration, environment integration, and multi-agent design patterns across real-world repositories. Our study addresses the following research questions:

- **RQ1:** Which algorithms are most prevalent in MARL implementations, and how frequently are multi-agent variants adopted compared to single-agent baselines?
- **RQ2:** How consistent are hyperparameter configurations across MARL implementations, and which parameters show consensus versus high variability?
- **RQ3:** Which recurring combinations of algorithms, libraries, environments, and multi-agent patterns appear across MARL repositories?

2 Background and Related Work

Prior empirical work has shown that implementation details in reinforcement learning can be as consequential as algorithmic innovations. In single-agent deep RL, Engstrom et al. [6] showed that code-level choices such as normalization, gradient clipping, and initialization can substantially affect performance, while Andrychowicz et al. [1] identified the impact of specific design and hyperparameter choices through large-scale empirical analysis. Similar observations have been reported in MARL. Papoudakis et al. [18] found that independent learners can remain competitive in some settings, Yu et al. [27] showed that implementation details such as value normalization and epoch tuning are critical to MAPPO performance, and Christianos et al. [5] demonstrated that the effectiveness of parameter sharing depends strongly on the environment. Framework and benchmark papers have defined much of the current MARL implementation landscape. RLlib [14], PettingZoo [26], MARLLib [11], BenchMARL [3], and JaxMARL [22] provide abstractions for multi-agent training, policy mapping, parameter sharing, and environment standardization, while platforms such as SMAC [23] shape how MARL systems are built and evaluated. However, these works primarily describe framework capabilities, benchmark ecosystems, and recommended ab-

stractions rather than how practitioners actually combine algorithms, libraries, environments, and multi-agent patterns across open-source implementations.

Complementary software engineering and tooling studies have examined practical challenges in RL system development. Henderson et al. [9] and Liaw et al. [15] highlighted reproducibility challenges and the importance of hyperparameter optimization infrastructure, Ntentos et al. [16] studied architectural design patterns in RL systems, and Fu et al. [8] revisited common practices in cooperative MARL. These studies provide important insights into RL engineering, but they do not characterize engineering decisions across a broad dataset of open-source MARL repositories. Our work addresses this gap by examining how MARL systems are engineered in practice, including algorithm adoption, configuration, environment and library use, and multi-agent structure.

3 Research Methodology

We analyzed 187 public GitHub repositories implementing MARL, comprising 12,205 files (9,957 Python, 2,090 YAML, and 158 JSON). Repositories were identified through GitHub Search API queries using broad MARL-related text and topic terms, while excluding algorithm and framework-specific search terms to reduce retrieval bias. The discovery queries included terms such as *marl*, variants of *multi-agent reinforcement learning*, and related GitHub topic labels. The full query list is included in the replication package.

Candidates were filtered in two stages. First, automated filters excluded forks, archived repositories, and non-Python repositories; we restricted the dataset to Python repositories because our detector pipeline relies on Python-specific static analysis, including AST parsing. We further removed repositories whose topics indicated coursework or tutorial content (e.g., tutorial, course, homework, thesis, demo), repositories whose name, description, or topics did not indicate both a multi-agent and a reinforcement-learning context, and repositories with fewer than 2 contributors or fewer than 10 commits to exclude toy and single-developer projects [12]. We retained repositories with activity after 1 October 2023. Prior work has used 12-month [2] and 24-month [13] thresholds to identify inactive projects. We chose this cutoff to balance inclusion of maintained but infrequently updated repositories with a focus on recent MARL practice. The dataset snapshot used in this study was collected in April 2026. Second, we manually screened the remaining repositories to retain only those that implement or apply MARL. We inspected each repository’s README, directory structure, source files, and configuration files, retaining repositories that contained executable code or configurations for training, evaluating, or applying RL with multiple agents. We excluded repositories that only mentioned MARL in documentation, collected paper lists, provided environments without agents, implemented general RL frameworks without a concrete MARL application, corresponded to core RL/MARL frameworks or libraries (e.g., Stable-Baselines3, RLLib, CleanRL, PettingZoo), or matched search terms without implementing MARL. Manual screening was conducted by the first author; ambiguous cases

Table 1. Indicators used by the multi-agent detector.

Indicator	Detection intuition
Instantiation of multiple agents	Code explicitly creates or configures more than one agent through constructor parameters or assignments.
Multiple agent classes	A module defines two or more classes whose names contain <i>Agent</i> , suggesting distinct agent roles or behaviors.
Multi-agent modules/imports	Imports from modules or classes whose names indicate multi-agent functionality (e.g., <i>MultiAgentEnv</i>), suggesting use of specialized multi-agent utilities.
Parallel environment detected	Assignments to names such as <i>VecEnv</i> or variables containing <i>parallel</i> , indicating vectorized or parallelized environment execution.
Multiple buffers	Imports of two or more classes whose names end in <i>buffer</i> , suggesting separate replay or storage buffers.
Multiple policies	Imports of more than one symbol containing <i>Policy</i> , suggesting management of multiple policy networks.
Multi-agent environment references	Assignments to <i>env_name</i> or <i>env_id</i> whose string values contain <i>multi</i> , suggesting a multi-agent environment variant.

were revisited after the initial pass and assessed against the same inclusion and exclusion criteria. This yielded 232 candidate repositories, of which 187 remained after screening. Metadata for all included and excluded repositories is available in the replication package.

The detailed algorithm-level analysis as seen in Section 4 was data-driven rather than based on a predefined selection of algorithms. We first applied the algorithm detector to the full repository dataset and then focused the analysis on the most prevalent detected algorithm families. The main analysis therefore covers PPO, DQN, Q-Learning, MAPPO and QMIX, which were the five most frequently observed families in the dataset. Less frequent algorithms and variants are retained in the repository-level variant summaries, but are not analyzed with the same level of detail to avoid overinterpreting patterns supported by only a smaller number of repositories.

To study engineering decisions directly from source code and configuration files, we implemented five static detectors for algorithm usage, environment setup, hyperparameter definitions, library usage, and multi-agent patterns. Using parsed Python files and YAML configurations, the detectors identify algorithms, environment setup patterns, hyperparameters, and RL libraries. Multi-agent pattern detection specifically relies on the seven indicators listed in Table 1.

4 Results

This section presents results on algorithm adoption and configuration, environment setup and integration, and multi-agent design patterns. We focus on usage patterns, frequency distributions, and cross-repository variation. The data are available in the replication package (Section 8).

Table 2. Environment prevalence across the 126 repositories in which at least one environment was detected (repositories may use multiple environments; % = share of those 126 repositories)

Category	Environment	Repos	%	Category	Environment	Repos	%
General-purpose	Gymnasium	65	51.6	Game-based	Atari	12	9.5
	PettingZoo	58	46.0		GridWorld	19	15.1
	SMAC/SMACv2	38	30.2		OpenSpiel	20	15.9
	MPE	40	31.7		GFootball	10	7.9
Cooperative MARL	Overcooked	10	7.9	Cont. control	MuJoCo	16	12.7
	RWARE	9	7.1		MaMuJoCo	9	7.1
	VMAS	6	4.8	Domain-specific	SUMO	10	7.9
	MeltingPot	5	4.0		MiniGrid	4	3.2
	LBF	7	5.6				
	Hanabi	3	2.4				

4.1 Environment Setup and Integration

Repository-level environment definitions can be grouped by primary backend or environment family. Table 2 summarizes their distribution across the 126 repositories in which at least one environment was detected. Gymnasium is the most common backend, appearing in 51.6% of these repositories (65/126; 95% Wilson CI [42.9, 60.1]). Among MARL-specific environments, PettingZoo appears in 46.0% of repositories (58/126; 95% CI [37.6, 54.7]), while SMAC/SMACv2 and MPE appear in 30.2% (38/126; 95% CI [22.8, 38.7]) and 31.7% (40/126; 95% CI [24.3, 40.3]) of repositories, respectively. Full 95% Wilson confidence intervals for repository-level prevalence proportions are provided in the replication package.

Figure 1 and Table 3(a) show how the five most prevalent algorithms (*PPO*, *DQN*, *Q-Learning*, *MAPPO*, and *QMIX*) are implemented across libraries in our dataset. **PyTorch** is the most widely used library across all five algorithm families, appearing in 65.4%-81.7% of repositories per algorithm, with the lowest share observed for MAPPO (65.4%). Beyond PyTorch, **Ray/RLlib** shows non-trivial adoption for PPO (24.5%) and MAPPO (15.4%) implementations. **Stable-Baselines3** appears primarily alongside PPO and DQN and is absent from QMIX implementations entirely. In contrast, **TensorFlow** appears less consistently and is concentrated in specific algorithms, with the highest share for DQN (15%) and QMIX (13.6%). **JAX** shows comparable adoption to TensorFlow in QMIX (13.6% each). Gymnasium and PettingZoo are included in the table as environment libraries commonly used with MARL algorithm implementations, rather than as training frameworks.

As shown in Figure 1, we examined temporal shifts in library adoption by grouping repositories into contiguous 2-year creation-year bins and computing, for each bin, the share of repositories in which a library was detected in at least one file. We use three bins: 2020-2021 ($n = 18$), 2022-2023 ($n = 45$), and 2024-2025 ($n = 73$), and omit 2018-2019 due to a small cohort ($n = 5$). Under this definition, adoption of PyTorch rises from 83.3% in 2020-2021 to 90.7% in 2024-2025, whereas TensorFlow declines from 27.8% to 8%. Because bin sizes are modest and proportions are not mutually exclusive, we report per-bin n and

Table 3. Repository-level summaries of algorithm, library, environment, and variant usage. In (a), percentages are computed over repositories where the column algorithm was detected (n in parentheses). Counts in (c) are non-exclusive, since repositories may implement multiple variants within a family.

(a) Algorithm-library distribution across repositories					
Library	PPO ($n=110$)	DQN ($n=60$)	Q-Learning ($n=43$)	MAPPO ($n=52$)	QMIX ($n=44$)
PyTorch	83 (75.5%)	49 (81.7%)	32 (74.4%)	34 (65.4%)	34 (77.3%)
Gymnasium	46 (41.8%)	27 (45.0%)	12 (27.9%)	20 (38.5%)	10 (22.7%)
Ray/RLlib	27 (24.5%)	9 (15.0%)	2 (4.7%)	8 (15.4%)	4 (9.1%)
PettingZoo	18 (16.4%)	3 (5.0%)	0 (0.0%)	5 (9.6%)	0 (0.0%)
SB3	19 (17.3%)	7 (11.7%)	1 (2.3%)	1 (1.9%)	0 (0.0%)
TensorFlow	4 (3.6%)	9 (15.0%)	5 (11.6%)	1 (1.9%)	6 (13.6%)
JAX	11 (10.0%)	3 (5.0%)	5 (11.6%)	6 (11.5%)	6 (13.6%)

(b) Algorithm-environment co-occurrence						
Algorithm	Gymnasium	PettingZoo	MPE	SMAC	OpenSpiel	GridWorld
PPO	41	34	31	26	15	12
DQN	24	25	15	14	9	9
Q-Learning	18	15	21	22	12	8
MAPPO	26	21	26	22	10	6
QMIX	21	19	25	26	11	4

(c) Repository-level prevalence of variants within selected algorithm families					
Family	Variant	Repos	Family	Variant	Repos
DDPG	DDPG	33	PPO	PPO	110
DDPG	MADDPG	32	PPO	MAPPO	52
DQN	DQN	60	PPO	IPPO	31
DQN	DDQN	13	PPO	HAPPO	10
DQN	Double DQN	12	PPO	VDPPO	2
DQN	Rainbow	10	QMIX	QMIX	44
DQN	Dueling DQN	8	QMIX	VDN	34
DQN	DRQN	6	QMIX	WQMIX	5
DQN	QRDQN	6	SAC	SAC	36

95% Wilson confidence intervals to communicate uncertainty in these estimates.

4.2 Algorithm Selection and Configuration

This subsection examines which algorithm families practitioners adopt, how variants are distributed within those families, and where hyperparameter choices converge or diverge. The five most prevalent algorithms in our dataset are PPO (110 repositories), DQN (60), MAPPO (52), QMIX (44), and Q-Learning (43). Percentages in this subsection are computed over configuration files within the corresponding algorithm family.

High consensus: QMIX shows the strongest hyperparameter consistency among the five algorithm families. As shown in Table 7, *double_q=True* is the only key exceeding 85% prevalence (85.9%). *target_update_interval=200* (80.8%), *buffer_size=5,000* (80.8%), and *mixing_embed_dim=32* (81.8%) also show strong, though slightly lower, consensus.

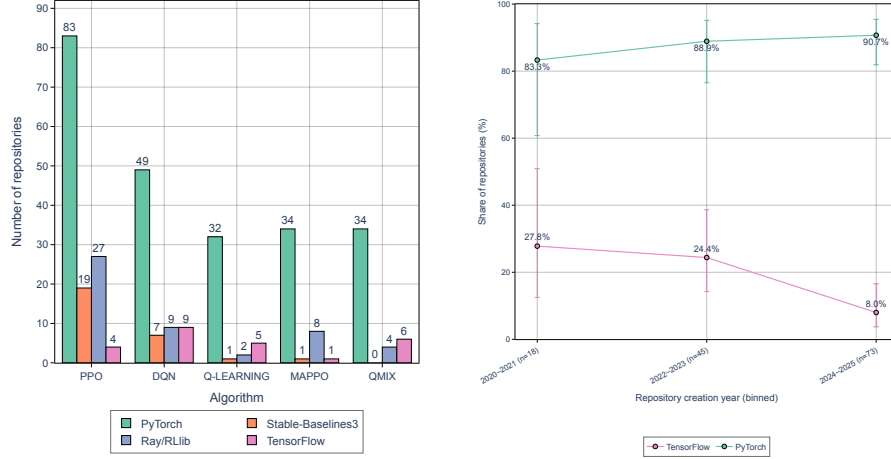


Fig. 1. RL library adoption patterns: algorithm-library usage (left) and PyTorch vs. TensorFlow adoption by repository creation year (right).

Table 4. Common PPO hyperparameter values across 395 YAML files.

Hyperparameter	Most frequent	2nd most frequent	3rd most frequent
gae_lambda	0.95 (274, 69.4%)	0.9 (3, 0.8%)	0.8 (1, 0.3%)
discount_factor	0.99 (228, 57.7%)	0.98 (21, 5.3%)	0.95 (16, 4.1%)
learning_rate	0.0007 (150, 38%)	0.0005 (45, 11.4%)	0.0004 (41, 10.4%)
activation	relu (184, 46.6%)	leaky_relu (47, 11.9%)	tanh (11, 2.8%)
vf_coef	1.0 (137, 34.7%)	0.25 (59, 14.9%)	0.5 (52, 13.2%)
ent_coef	0.01 (220, 55.7%)	0.0 (28, 7.1%)	0.000002 (1, 0.3%)

Moderate consensus: PPO and MAPPO show partial convergence around a small set of recurring hyperparameters. In PPO, the clearest recurring values are *gae_lambda*=0.95 (69.4%) and *ent_coef*=0.01 (55.7%) (Table 4). MAPPO (Table 5) shows a similar pattern, with recurring use of *discount_factor* and *activation*, while still varying in architectural choices such as representation (*Basic_RNN*: 39.5%, *Basic_MLP*: 26.1%). Taken together, these results suggest partial standardization around a limited set of defaults rather than broad agreement across all hyperparameters.

Low consensus: DQN and Q-Learning exhibit the lowest degree of standardization. As shown in Table 6, both families vary substantially in *buffer_size*, *target_update_interval*, and *learning_rate*. Although *discount_factor* is the most stable hyperparameter in Q-Learning, its most common value, 0.99, still appears in only 57.6% of files, indicating limited value-level convergence.

Tables 8 and 9 summarize hyperparameter variability across algorithm families using the coefficient of variation (CV) for continuous parameters and normalized Shannon entropy for discrete ones. Among continuous parameters, *dis-*

Table 5. Common MAPPO hyperparameter values across 2184 YAML files.

Hyperparameter	Most frequent	2nd most frequent	3rd most frequent
learning_rate	0.0007 (531, 38.9%)	0.001 (194, 14.2%)	0.0004 (102, 7.5%)
discount_factor	0.99 (1010, 73.9%)	0.95 (129, 9.4%)	0.98 (105, 7.7%)
representation	Basic_RNN (540, 39.5%)	Basic_MLP (357, 26.1%)	Basic_Idential (143, 10.5%)
activation	relu (912, 66.8%)	leaky_relu (240, 17.6%)	tanh (11, 0.8%)
running_steps	10000000 (376, 27.5%)	1000000 (248, 18.2%)	2000000 (138, 10.1%)
buffer_size	5000 (297, 21.7%)	128 (169, 12.4%)	100000 (147, 10.8%)
use_grad_clip	False (668, 48.9%)	True (463, 33.9%)	–
grad_clip_norm	0.5 (821, 60.1%)	10 (296, 21.7%)	100.0 (12, 0.9%)
use_actions_mask	False (560, 41%)	True (444, 32.5%)	–

Table 6. Common DQN and Q-Learning hyperparameter values across 92 and 175 YAML files.

Hyperparameter	DQN		Q-Learning	
	Most frequent	2nd most frequent	Most frequent	2nd most frequent
buffer_size	200000 (50, 28.6%)	100000 (32, 18.3%)	5000 (47, 51.1%)	100000 (8, 8.7%)
target_update_interval	500 (50, 28.6%)	100 (47, 26.9%)	200 (47, 51.1%)	100 (9, 9.8%)
learning_rate	0.001 (90, 51.4%)	0.0001 (53, 30.3%)	0.0007 (34, 37.0%)	0.001 (13, 14.1%)
discount_factor	0.99 (148, 84.6%)	0.9 (2, 1.1%)	0.99 (53, 57.6%)	0.9 (8, 8.7%)

count_factor is the only consistently stable hyperparameter across all five families, with CV values below 0.06; *gae_lambda* is similarly stable in PPO and MAPPO, with CV values below 0.08. In contrast, *learning_rate*, *buffer_size*, *batch_size*, and *running_steps* show substantial variability, indicating limited convergence beyond a small set of shared defaults. Among discrete parameters, *policy*, *use_actions_mask*, and *agent* remain highly variable in several families. *use_grad_clip* is fully standardized in DQN, PPO, and Q-Learning, but shows near-maximum disagreement in MAPPO (0.976), indicating that gradient clipping practice is not standardized across MAPPO implementations. *double_q* reaches full consensus across all families where it appears. *use_rnn* shows moderate variability in most families but crosses into high variability in Q-Learning (0.811). *representation* shows moderate variability throughout, suggesting only partial convergence on architectural choices. Figure 2 shows substantial variation in the median number of hyperparameter keys per configuration file, ranging from 3 to 42 across algorithms with at least 15 configuration files. The highest medians occur in *WQMIX* (42), *PDQN* (33), and *Double DQN* (31), whereas simpler baselines such as *FACMAC* (3), *DDPG* (6), and *SAC* (6) remain in the single digits. More specialized variants also tend to require larger hyperparameter sets than their base algorithms: *DQN* has a median of 9 keys, compared with 25-31 across its distributional variants; *PPO* increases from 10 to 12 in *IPPO* and 11 in *MAPPO*; and *QMIX* (15) and *VDN* (9) exceed Q-Learning (7). Overall, this suggests that configuration burden tends to increase with algorithm specialization.

Table 7. Top 3 values for common hyperparameter keys for QMIX.

Hyperparameter	Most frequent	2nd most frequent	3rd most frequent
buffer_size	5000 (160, 80.8%)	100000 (13, 6.6%)	2000 (4, 2.0%)
target_update_interval	200 (160, 80.8%)	100 (11, 5.6%)	10 (1, 0.5%)
double_q	True (170, 85.9%)	—	—
mixing_embed_dim	32 (162, 81.8%)	128 (13, 6.6%)	64 (8, 4.0%)
hypernet_embed	64 (105, 53.0%)	32 (60, 30.3%)	128 (12, 6.1%)
running_steps	10000000 (31, 30.7%)	1000000 (18, 17.8%)	2000000 (12, 11.9%)

Table 8. Hyperparameter variability across algorithm families (CV for continuous parameters; $n \geq 15$).

Hyperparameter	DQN	MAPPO	PPO	Q-Learning	QMIX
High consensus (CV < 0.1)					
discount_factor	0.010	0.014	0.055	0.031	0.007
gae_lambda	—	0.075	0.011	—	—
High variability (CV > 0.4)					
learning_rate	5.261	5.501	0.534	1.172	0.479
buffer_size	1.007	4.673	2.182	2.758	3.062
batch_size	0.682	1.320	3.483	1.608	1.765
running_steps	2.003	2.056	1.087	1.057	0.950

4.3 Multi-Agent Practices

We report repository-level prevalence of multi-agent implementation indicators across the five algorithm families and their variants (Figure 3). Each heatmap cell shows the number of repositories in which an algorithm or variant exhibits a given indicator. Across base algorithms, *Instantiation of multiple agents* and *Multi-agent modules or classes found* show the highest prevalence. For example, *Instantiation of multiple agents* appears in 84.5% of PPO repositories (93/110; 95% Wilson CI [76.6, 90.1]). Other indicators occur less frequently, including *Multiple agent classes found* (25-45 repositories), *Multiple buffers found* (10-17), *Multiple policies found* (8-19), and *Parallel environment detected* (6-15). Variants follow the same relative ordering as their base algorithms but at lower absolute counts: among PPO variants, IPPO shows the highest counts (29 repositories for *Instantiation of multiple agents*), while HAPPO and VDPPO appear far less frequently. In the DQN family, Double DQN leads (15 repositories), and among Q-Learning variants, IQL (27) exceeds Double Q-Learning (21). WQMIX appears in substantially fewer repositories than QMIX (5 versus 43 for *Instantiation of multiple agents*). *Multiple buffers found* consistently shows the lowest prevalence across all families. Full 95% Wilson confidence intervals are provided in the replication package.

5 Validation

We validated five detectors: Multi-agent categorization, algorithm, library, environment, and hyperparameter detectors. Validation samples were generated from the detector outputs using scripts included in the replication package. For

Table 9. Discrete hyperparameter variability across algorithm families (normalized Shannon entropy; 0 = full consensus, 1 = maximum disagreement; $n \geq 15$).

Hyperparameter	DQN	MAPPO	PPO	Q-Learning	QMIX
High variability (entropy > 0.8)					
policy	0.881	0.885	0.998	0.000	0.993
use_actions_mask	0.000	0.990	0.995	0.439	0.622
agent	0.878	0.876	0.666	0.258	0.486
use_grad_clip	0.000	0.976	0.000	0.000	0.107
use_rnn	–	0.662	0.684	0.811	0.582
Moderate variability ($0.1 < \text{entropy} \leq 0.8$)					
representation	0.616	0.615	0.652	0.555	0.378
activation	0.145	0.387	0.607	0.000	0.000
use_parameter_sharing	–	0.358	0.689	0.000	0.253
Complete consensus (entropy = 0)					
double_q	0.000	0.000	–	0.000	0.000

Table 10. Environment detector validation

Level	Value	What was checked
Instance-level (detector-reported occurrences)		
Sampled instances	100	(repo, file, occurrence) outputs flagged by detector
TP / FP	75 / 25	True env instantiation/wrapping vs. non-env matches
Precision	0.750	95% CI: [0.657, 0.825]
Repository-level (coverage of env setup)		
Human-positive repositories	67	Repos manually confirmed to contain env setup code
TP / FN	57 / 10	Repo detected vs. repo missed
Repo-level recall	0.851	95% CI: [0.747, 0.917]

Note: Precision = $TP/(TP + FP)$ over sampled instances; repo-level recall = $TP/(TP + FN)$ over repositories manually confirmed to contain env setup code.

instance-level precision, we sampled detected occurrences for the algorithm, library, environment, and MARL-indicator detectors; for hyperparameters, we sampled detected configuration files. To reduce sampling bias towards large repositories, instance/file-level samples were stratified by repository. For recall, we sampled repositories or files from the dataset and manually checked whether the corresponding construct was present but missed by the detector. Each sampled item was manually inspected against the surrounding source-code or configuration context. A true positive required that the detected evidence correspond to the intended construct, rather than appearing only in comments, documentation, unrelated variable names, generic configuration files, or non-training artifacts. Validation was conducted by the first author, with ambiguous cases revisited after the initial pass. We report precision with 95% Wilson confidence intervals and, where feasible, recall at the repository or file level. Results are summarized in Tables 10 and 11; complete validation samples and scripts are provided in the replication package.

Environment detector. Of 100 reported instances, 75 were true positives (precision = 0.750, 95% CI [0.657, 0.825]). Among 67 repositories manually confirmed to contain an environment setup, 57 were detected (recall = 0.851, 95% CI [0.747,

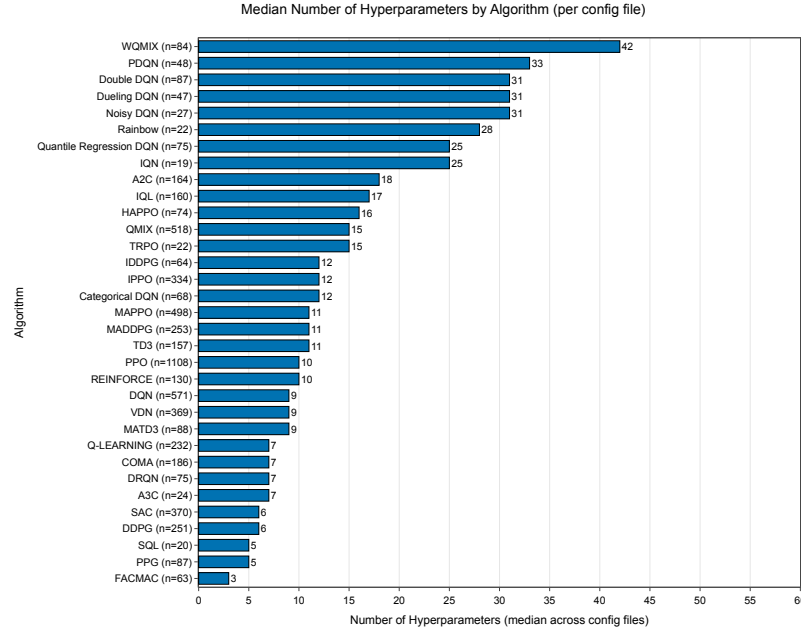


Fig. 2. Median number of hyperparameters by algorithm (algorithms with $n \geq 15$ configuration files).

0.917]). The false negatives mainly concerned non-standard, domain-specific instantiation patterns or repositories with minimal executable code.

Algorithm and library detectors. Manual checks confirm an algorithm detector precision of 0.890 (89/100; 95% CI [0.814, 0.937]) and repository-level recall of 0.953 (61/64; 95% CI [0.871, 0.984]). Its false positives arose from clues based solely on names or from closely related variants assigned canonical names. The library detector achieved instance-level precision of 1.000 (100/100; 95% CI [0.963, 1.000]) and repository-level recall of 1.000 (69/69; 95% CI [0.947, 1.000]).

MARL detector. The classifier achieves repository-level precision of 0.895 and recall of 0.785; MARL indicators yield instance-level precision of 0.760 (76/100; 95% CI [0.668, 0.833]) and repository-level recall of 0.896 (69/77; 95% CI [0.808, 0.946]) (full CIs in Table 11). False positives stem from non-MARL agentic tools; false negatives from multi-agent structures implemented via indirect wrappers rather than standard APIs.

Hyperparameter detector. Across 100 configuration files, precision was 0.890 (89/100; 95% CI [0.814, 0.937]) and file-level recall was 0.986 (71/72; 95% CI [0.925, 0.998]). The false positives included generic CI YAML files and non-training configuration files.

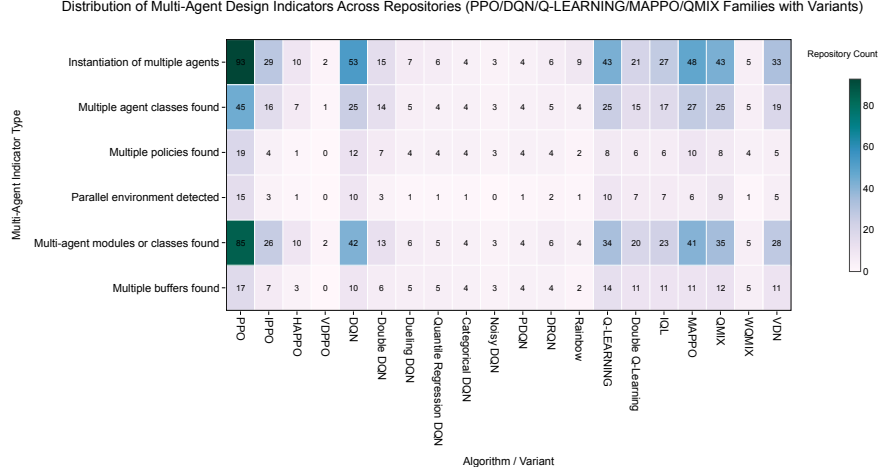


Fig. 3. Distribution of multi-agent indicators across RL algorithms.

6 Discussion

This section interprets our findings in relation to the research questions from Section 1 and discusses implications for practitioners and researchers.

6.1 RQ1: Algorithm Prevalence and Variant Adoption

RQ1 examined which algorithm families are most prevalent in open-source MARL repositories and how variant adoption appears within these families. The five most prevalent algorithms in our dataset are PPO (110 repositories), DQN (60), MAPPO (52), QMIX (44), and Q-Learning (43). Variant-level counts in Table 3 show that the relationship between base algorithms and their variants differs across families: PPO remains prevalent but coexists with substantial MAPPO and IPPO usage, DQN variants are more fragmented, DDPG and MADDPG show nearly equal repository prevalence, and the QMIX family includes both QMIX and VDN as strongly represented methods. These patterns indicate that variant adoption is family-specific: some families remain centered on a widely reused baseline, whereas others are organized around several specialized variants that are themselves reused across repositories. More specialized variants also tend to involve larger hyperparameter sets than their base algorithms (Figure 2). DQN has a median of 9 hyperparameter keys, whereas distributional DQN-style variants reach 25-31; PPO increases from 10 to 12 in IPPO and 11 in MAPPO; and QMIX (15) and VDN (9) exceed Q-Learning (7).

For practitioners. Widely reused families such as PPO and QMIX provide practical starting points, but adapting single-agent baselines to multi-agent settings

Table 11. Validation results for detection systems

Detector	Level	n	TP	FP	Precision [95% CI]
Algorithm	Instance	100	89	11	0.890 [0.814, 0.937]
Library	Instance	100	100	0	1.000 [0.963, 1.000]
Hyperparameter	File	100	89	11	0.890 [0.814, 0.937]
MARL indicators	Instance	100	76	24	0.760 [0.668, 0.833]
Detector	Level	n	TP	FN	Recall [95% CI]
Algorithm	Repository	64	61	3	0.953 [0.871, 0.984]
Library	Repository	69	69	0	1.000 [0.947, 1.000]
Hyperparameter	File	72	71	1	0.986 [0.925, 0.998]
MARL indicators	Repository	77	69	8	0.896 [0.808, 0.946]
Detector	Level	n	TP/FP	TN/FN	Precision / Recall [95% CI]
MARL classifier	Repository	80	51/6	9/14	0.895 [0.789, 0.951] / 0.785 [0.670, 0.867]

Note: Wilson CIs throughout. Precision validated at instance/file level; recall measured against human-labeled positive sets at repo/file level.

through configuration changes alone may not capture important architectural differences, and practitioners should verify that such adaptations match their requirements.

For researchers. The lower adoption of multi-agent-specific algorithms relative to single-agent counterparts highlights an important question for future work: whether implementation complexity and configuration burden hinder reuse. Investigating what drives variant adoption remains an important direction, particularly in families where no single variant has achieved broad community reuse.

6.2 RQ2: Consistency and Variability of Hyperparameters

RQ2 examined the extent to which MARL repositories converge on common hyperparameter conventions. Across the five most prevalent algorithm families, only a narrow shared core of hyperparameter choices appears consistently reused across repositories: *discount_factor*=0.99 is stable throughout, and *gae_lambda* shows similarly strong convergence where applicable. Beyond this small core, however, hyperparameter practice is fragmented and strongly family-dependent. QMIX exhibits the strongest standardization, with *double_q*=True as the only key exceeding 85% prevalence (85.9%) and several others clustered just below, whereas PPO and MAPPO show only partial convergence around a limited set of recurring defaults. In contrast, DQN and Q-Learning display the weakest standardization, with substantial variation in training-scale and optimization parameters such as *buffer_size*, *batch_size*, and *learning_rate*. Tables 8 and 9 reinforce this pattern: a small number of parameters behave like widely reused defaults, while most hyperparameter choices remain highly variable across repositories. These patterns suggest that open-source MARL development relies on only a limited set of hyperparameter conventions, with much of the remaining tuning logic encoded in family-specific or project-specific choices rather than broadly shared practice.

For practitioners. Parameters with high consensus (e.g., `discount_factor=0.99`, `gae_lambda=0.95`) may serve as reasonable starting points, while high-variability parameters such as buffer sizes, learning rates, and running steps likely require domain-specific tuning. More advanced algorithms also tend to involve larger hyperparameter sets, increasing development and debugging effort.

For researchers. The limited standardization of hyperparameters beyond a few recurring defaults highlights the need for systematic studies of sensitivity and transferability across MARL settings. Evidence-based configuration guidance could help separate reusable defaults from task or algorithm-specific parameters and reduce trial and error in open-source MARL development.

6.3 RQ3: Recurring Combinations of Engineering Decisions

RQ3 examined which engineering decisions in MARL repositories recur together across algorithms, libraries, environments, and multi-agent patterns. PyTorch is the most widely used library across all five algorithm families, while other libraries appear in more family-specific combinations. Algorithm-environment pairings similarly show consistency rather than independent choices: PPO co-occurs frequently with Gymnasium, PettingZoo, SMAC, and MPE; DQN and Q-Learning are often paired with Gymnasium, PettingZoo, and SMAC-based environments; and QMIX co-occurs most strongly with SMAC. At the structural level, the most prevalent multi-agent indicators across families are *Instantiation of multiple agents* and *Multi-agent modules or classes found*, while more specialized patterns such as *Multiple policies found* and *Multiple buffers found* occur less frequently. These results suggest that engineering choices tend to appear in recurring combinations rather than as isolated decisions. Some pairings are concentrated around established benchmark ecosystems, such as QMIX with SMAC. The higher prevalence of simple structural patterns indicates that repositories in our dataset more often realize multi-agent behavior through straightforward implementation structures than through complex coordination-specific ones.

For practitioners. When adopting an algorithm, practitioners may benefit from examining which libraries, environments, and structural patterns commonly accompany it in existing repositories (Table 3, Figure 3). These recurring combinations can serve as practical reference points, though not as universally appropriate solutions.

For researchers. The recurring combinations of engineering decisions in our dataset suggest that studies of individual factors, such as algorithms or hyperparameters, should account for implementation context, including library, environment, and system structure. The limited use of specialized multi-agent architectures also raises the question of whether their added complexity yields practical benefits.

7 Threats to Validity

Internal Validity. The dataset was compiled using GitHub search queries and broad MARL-related filtering criteria; repositories using unconventional names, descriptions, or topic labels may have been missed. We mitigated this by using diverse search terms, applying consistent inclusion and exclusion criteria, manually screening candidate repositories, and documenting repository metadata in the replication package. The resulting dataset should therefore be interpreted as a broad but not exhaustive sample of observable open-source MARL repositories.

External Validity. This study focuses on open-source MARL repositories on GitHub, so the findings primarily reflect engineering practices visible in public repositories. Proprietary or industrial MARL systems may follow different development processes, quality standards, and deployment constraints. The analyzed repositories also vary in maturity, maintenance activity, and intended purpose. A detailed stratification by repository quality or production readiness is outside the scope of this study; hence, our analysis focuses on broad engineering decisions observable in public MARL repositories at the time of data collection.

Construct Validity. Our static detectors may not fully capture design intent. This is most relevant for multi-agent indicators, where terms such as *Agent*, *Policy*, *Buffer*, or *multi* may reflect naming conventions or partial implementations rather than deliberate architectural choices. However, these indicators characterize structural patterns within repositories already identified as MARL implementations during manual screening; they are not used to classify repositories. We mitigated this risk through stratified manual validation and report precision, recall, and confidence intervals throughout. Since the environment and MARL-indicator detectors have lower precision than the remaining detectors, findings on environment usage and multi-agent structures should be interpreted as approximate prevalence estimates rather than exact measurements.

Validity of Conclusions. This study provides a descriptive empirical baseline rather than causal evidence on which practices lead to better performance. Some analyses are reported at the repository level while others, particularly hyperparameter summaries, are reported at the file level; repositories with more configuration files contribute proportionally more to file-level counts. We make the unit of analysis explicit in each table and interpret file-level results as configuration patterns in the analyzed dataset rather than as independent developer decisions. Future work could complement these findings with controlled experiments evaluating the impact of specific configuration choices on MARL performance.

8 Conclusion

Our study provides an empirical characterization of engineering decisions across 187 open-source MARL repositories. Open-source MARL development is concentrated around a small number of algorithm families, especially *PPO*, *DQN*, *QMIX*, and their variants. Hyperparameter standardization remains limited beyond a few recurring defaults, with substantial variation across repositories and

a higher configuration burden in more specialized variants. Multi-agent behavior is more often realized through simple structural patterns such as agent instantiation and multiple agent classes than through specialized coordination abstractions, algorithms, libraries, or environments, and algorithms, libraries, environments, and multi-agent patterns co-occur in recurring combinations rather than appearing independently. These results provide an empirical baseline of current MARL practice and highlight opportunities for stronger standardization, improved tooling, and controlled studies of the impact of configuration on performance.

Acknowledgments. This work was supported by: FFG (Austrian Research Promotion Agency) project MODIS (no. FO999895431); project BEAM (no. FO999933314).

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

Data Availability. The replication package is available at: <https://doi.org/10.5281/zenodo.19656029>.

References

1. Andrychowicz, M., Raichuk, A., Stańczyk, P., Orsini, M., Girgin, S., Marinier, R., Hussenot, L., Geist, M., Pietquin, O., Michalski, M., Gelly, S., Bachem, O.: What matters in on-policy reinforcement learning? a large-scale empirical study (2020), <https://arxiv.org/abs/2006.05990>
2. Avelino, G., Constantinou, E., Valente, M.T., Serebrenik, A.: On the abandonment and survival of open source projects: An empirical investigation. In: 2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM). pp. 1–12. IEEE (2019)
3. Bettini, M., Prorok, A., Moens, V.: Benchmarl: Benchmarking multi-agent reinforcement learning. *Journal of Machine Learning Research* **25**(217), 1–10 (2024)
4. Busoniu, L., Babuska, R., De Schutter, B.: A comprehensive survey of multiagent reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)* **38**(2), 156–172 (2008)
5. Christianos, F., Schäfer, L., Albrecht, S.V.: Shared experience actor-critic for multi-agent reinforcement learning (2021), <https://arxiv.org/abs/2006.07169>
6. Engstrom, L., Ilyas, A., Santurkar, S., Tsipras, D., Janoos, F., Rudolph, L., Madry, A.: Implementation matters in deep rl: A case study on ppo and trpo. In: International conference on learning representations (2019)
7. Foerster, J.N., et al.: Counterfactual multi-agent policy gradients. In: AAAI (2018)
8. Fu, W., Yu, C., Xu, Z., Yang, J., Wu, Y.: Revisiting some common practices in cooperative multi-agent reinforcement learning (2022), <https://arxiv.org/abs/2206.07505>
9. Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., Meger, D.: Deep reinforcement learning that matters. In: Proceedings of the AAAI conference on artificial intelligence. vol. 32 (2018)

10. Hernandez-Leal, P., Kartal, B., Taylor, M.E.: A survey and critique of multiagent deep reinforcement learning. *Autonomous Agents and Multi-Agent Systems* **33**(6), 750–797 (2019)
11. Hu, S., Zhong, Y., Gao, M., Wang, W., Dong, H., Liang, X., Li, Z., Chang, X., Yang, Y.: Marllib: A scalable and efficient multi-agent reinforcement learning library. *Journal of Machine Learning Research* **24**(315), 1–23 (2023)
12. Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D.M., Damian, D.: The promises and perils of mining github. In: *Proceedings of the 11th working conference on mining software repositories*. pp. 92–101 (2014)
13. Khondhu, J., Capiluppi, A., Stol, K.J.: Is it all lost? a study of inactive open source projects. In: *IFIP international conference on open source systems*. pp. 61–79. Springer (2013)
14. Liang, E., Liaw, R., Nishihara, R., Moritz, P., Fox, R., Goldberg, K., Gonzalez, J., Jordan, M., Stoica, I.: Rllib: Abstractions for distributed reinforcement learning. In: *International conference on machine learning*. pp. 3053–3062. PMLR (2018)
15. Liaw, R., Liang, E., Nishihara, R., Moritz, P., Gonzalez, J.E., Stoica, I.: Tune: A research platform for distributed model selection and training. *arXiv preprint arXiv:1807.05118* (2018)
16. Ntentos, E., Warnett, S.J., Zdun, U.: Supporting architectural decision making on training strategies in reinforcement learning architectures. In: *2024 IEEE 21st International Conference on Software Architecture (ICSA)*. pp. 90–100. IEEE (2024)
17. Orr, J., Dutta, A.: Multi-agent deep reinforcement learning for multi-robot applications: A survey. *Sensors* **23**(7), 3625 (2023)
18. Papoudakis, G., Christianos, F., Schäfer, L., Albrecht, S.V.: Benchmarking multi-agent deep reinforcement learning algorithms in cooperative tasks (2021), <https://arxiv.org/abs/2006.07869>
19. Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., Dormann, N.: Stable-baselines3: Reliable reinforcement learning implementations. *Journal of machine learning research* **22**(268), 1–8 (2021)
20. Rashid, T., et al.: Qmix: Monotonic value function factorisation for deep multi-agent reinforcement learning. In: *ICML*. vol. 80, p. 4295–4304 (2018)
21. Roesch, M., Linder, C., Zimmermann, R., Rudolf, A., Hohmann, A., Reinhart, G.: Smart grid for industry using multi-agent reinforcement learning. *Applied Sciences* **10**(19), 6900 (2020)
22. Rutherford, A., Ellis, B., Gallici, M., Cook, J., Lupu, A., Ingvarsson Juto, G., Willi, T., Hammond, R., Khan, A., Schroeder de Witt, C., et al.: Jaxmarl: Multi-agent rl environments and algorithms in jax. *Advances in Neural Information Processing Systems* **37**, 50925–50951 (2024)
23. Samvelyan, M., et al.: The starcraft multi-agent challenge. *CoRR* **abs/1902.04043** (2019)
24. Shalev-Shwartz, S., Shammah, S., Shashua, A.: Safe, multi-agent, reinforcement learning for autonomous driving. *arXiv preprint arXiv:1610.03295* (2016)
25. Sunehag, P., Lever, G., Gruslys, A., Czarnecki, W.M., Zambaldi, V., Jaderberg, M., Lanctot, M., Sonnerat, N., Leibo, J.Z., Tuyls, K., et al.: Value-decomposition networks for cooperative multi-agent learning. *arXiv preprint arXiv:1706.05296* (2017)
26. Terry, J.K., Black, B.J., et al.: Pettingzoo: Gym for multi-agent reinforcement learning. In: *NeurIPS Datasets and Benchmarks Track* (2021)
27. Yu, C., Velu, A., Vinitsky, E., Gao, J., Wang, Y., Bayen, A., Wu, Y.: The surprising effectiveness of ppo in cooperative, multi-agent games (2022), <https://arxiv.org/abs/2103.01955>